

Devoir 5 (H26)

MAT-2910 Analyse numérique pour l'ingénierie

Département de mathématiques et statistique de l'Université Laval

0 Consignes (Python)

- Vous pouvez former des équipes d'**au plus** 2 personnes. Il faut créer l'équipe sur monPortail avant la date limite indiquée (**même si on est seul!**).
- Vous pouvez être dans la même équipe que pour un devoir précédent.
- Vous n'avez pas droit à l'aide du CDA pour ces exercices. Vous pouvez utiliser le forum, demander à l'auxiliaire du cours ou à l'enseignant.
- Dans la boîte de dépôt de monportail, vous devez remettre un seul fichier : un dossier compressé **en format .zip** par équipe nommé `EquipeX.zip`, où X est votre numéro d'équipe. Ce dossier devra contenir un **rapport écrit au traitement de texte** (*e.g.* MS Word, L^AT_EX) **converti en format .pdf** contenant tout ce qui est demandé pour les exercices (calculs, figures, explications, valeurs numériques, etc.). Le dossier contiendra aussi les codes **au format .py** demandés.
- **Codes demandés** : Un script `equipeXX.py` où XX désigne votre numéro d'équipe, la fonction `rk4.py` fournie (**ne pas la modifier**), et la fonction `splines_edo_implicit.py` complétée.
- Le script doit générer toutes les figures demandées lorsqu'on l'exécute, et elles doivent concorder avec celles dans votre rapport. Dans le cas contraire, le code se verra attribuer la note de 0.
- Des points sont attribués à la présentation des figures (propreté, identification des axes, titres, légendes, couleurs). N'oubliez donc pas d'en tenir compte lorsque vous les incluez au rapport.
- Votre travail doit être remis sur la boîte de dépôt monPortail avant la date limite indiquée sur le site de cours. En cas de retard, une pénalité de 5%/heure s'applique lors des premières 24h, jusqu'à concurrence de 20%. Après la première journée de retard, 20%/jour supplémentaire sera enlevé.

1 Importations

Dans votre script `equipeXX.py`, vous devrez inclure les importations suivantes :

```
import numpy as np
import matplotlib.pyplot as plt
from splines_edo_implicite import splines_edo_implicite
from rk4 import rk4
```

2 Question 1

Soit un intervalle de temps $[a, b]$ découpé en N sous-intervalles $[t_i, t_{i+1}]$, où $t_0 = a$, $t_N = b$, $t_{i+1} = t_i + h$ et $h = (b - a)/N$. On cherche la solution y à un problème à valeur initiale (PVI) de second ordre,

$$\begin{cases} y''(t) = f(t, y(t), y'(t)), & t \in [a, b] \\ y(a) = \alpha \\ y'(a) = \beta \end{cases}$$

On a vu/verra en classe qu'on peut résoudre un problème de cette forme en le convertissant en un système d'équations du premier ordre. On présente ici une autre façon de traiter ce problème.

On va décider d'approximer y par un polynôme cubique par morceaux

$$y(t) \approx q_i(t) \quad \text{si } t \in [t_i, t_{i+1}],$$

où $q_i(t) = a_i t^3 + b_i t^2 + c_i t + d_i$ (les coefficients sont inconnus). Par conséquent,

$$\begin{aligned} y'(t) &\approx q'_i(t) = 3a_i t^2 + 2b_i t + c_i \quad \text{si } t \in [t_i, t_{i+1}], \\ y''(t) = f(t, y(t), y'(t)) &\approx q''_i(t) = 6a_i t + 2b_i \quad \text{si } t \in [t_i, t_{i+1}]. \end{aligned}$$

Pour que $q_0(t)$ respecte l'équation différentielle et les conditions initiales en t_0 et en t_1 , il doit satisfaire :

$$\begin{cases} q_0(t_0) = y(t_0) \\ q'_0(t_0) = y'(t_0) \\ q''_0(t_0) = f(t_0, y(t_0), y'(t_0)) \\ q''_0(t_1) = f(t_1, q_0(t_1), q'_0(t_1)) \end{cases} \iff \begin{cases} q_0(t_0) = \alpha \\ q'_0(t_0) = \beta \\ q''_0(t_0) = f(t_0, \alpha, \beta) \\ 6a_0 t_1 + 2b_0 = f(t_1, q_0(t_1), q'_0(t_1)) \end{cases} \quad (1)$$

Une fois développée, la dernière équation est

$$6a_0t_1 + 2b_0 - f(t_1, a_0t_1^3 + b_0t_1^2 + c_0t_1 + d_0, 3a_0t_1^2 + 2b_0t_1 + c_0) = 0$$

qui est en général non-linéaire (à cause de f). Il faut, dans ce cas, résoudre un système d'équations non-linéaires à 4 inconnues pour déterminer q_0 . Plus spécifiquement, il faudra résoudre

$$\vec{F}(a_0, b_0, c_0, d_0) = \vec{0} \iff \begin{cases} a_0t_0^3 + b_0t_0^2 + c_0t_0 + d_0 - \alpha = 0 \\ 3a_0t_0^2 + 2b_0t_0 + c_0 - \beta = 0 \\ 6a_0t_0 + 2b_0 - f(t_0, \alpha, \beta) = 0 \\ 6a_0t_1 + 2b_0 - f(t_1, a_0t_1^3 + b_0t_1^2 + c_0t_1 + d_0, 3a_0t_1^2 + 2b_0t_1 + c_0) = 0 \end{cases}$$

Une fois le polynôme q_0 trouvé, on cherchera q_1 . Pour déterminer ses coefficients, on supposera que

$$y(t_1) \approx q_0(t_1) \quad \text{et} \quad y'(t_1) \approx q'_0(t_1).$$

Cela nous permettra encore de poser un système de 4 équations et 4 inconnus à résoudre. On répètera cette méthodologie pour déterminer successivement les q_i sur chaque sous-intervalle.

2.1 Polynôme cubique par morceaux

- En s'inspirant de (1) et du système $\vec{F}(a_0, b_0, c_0, d_0) = \vec{0}$ pour déterminer q_0 , écrire les 4 équations à résoudre pour déterminer les coefficients de q_i , $i \geq 1$. Compléter le code de la fonction `spline_edo_implicite.py` pour y ajouter le système trouvé.
- À partir des équations posées, justifier que l'approximation de y (un polynôme de degré 3 par morceaux) donnée par cet algorithme est en fait une spline cubique. Dans quel cas est-elle naturelle ?
- Soit le PVI du second ordre concret suivant :

$$\begin{cases} y''(t) &= \frac{2e^{-t}}{\sin t} y(t) y'(t) - 2e^t \sin t, & t \in [\pi/4, 3] \\ y(\pi/4) &= \frac{\sqrt{2}}{2} e^{\pi/4}, \\ y'(\pi/4) &= \sqrt{2} e^{\pi/4}. \end{cases} \quad (2)$$

Dans votre script, faire appel à `spline_edo_implicite.py` afin de résoudre le PVI ci-haut, en utilisant $N = 16$ polynômes cubiques. Utilisez les coefficients trouvés afin de représenter la spline cubique $S(t)$ ainsi que la solution exacte au PVI

$$y(t) = e^t \sin t,$$

toutes les deux sur la même figure (Figure 1). Commenter.

- d) Dans votre script, utilisez $N = 2^6, 2^7, 2^8, 2^9, 2^{10}$ polynômes cubiques pour composer des splines comme approximations, puis tracez en échelle logarithmique l'erreur **globale** en fonction de $h = (b - a)/N$ sur une figure (Figure 2). Commenter.

Rappel : on définit l'erreur globale d'approximation à un PVI par

$$E(h) = \max_{0 \leq i \leq N} |y(t_i) - S_h(t_i)|$$

où $S_h(t)$ est la spline cubique créée en utilisant des sous-intervalles de longueur h pour approximer $y(t)$.

- e) Est-ce que la méthode d'approximation par spline cubique converge ? Si oui, à quel ordre ? Utilisez votre Figure 2 pour justifier.
- f) On a effectué le choix d'utiliser des splines cubiques comme approximation. Indiquer pourquoi le choix d'une spline linéaire ne peut pas toujours être utilisé dans ce contexte-ci. **Indice :** regarder la première équation du PVI et se rappeler des avantages d'une spline cubique par rapport à une spline linéaire.

2.2 Systèmes d'EDOs

- g) Dans le rapport, réécrire le PVI (2) comme un système de deux équations différentielles du premier ordre avec les conditions initiales appropriées.
- h) Dans le rapport, poser le calcul à faire si vous deviez trouver le premier pas de temps pour le système d'équations différentielles précédent en utilisant la méthode d'Euler explicite (ne pas calculer le résultat).
- i) À l'aide de la fonction `rk4.py` fournie, résoudre par Runge-Kutta 4 le système d'EDO du premier ordre déterminé en g) en utilisant $N = 16$ sous-intervalles. Afficher sur une même figure (Figure 3) la solution exacte $y(t)$, la spline cubique $S(t)$ obtenue en c), ainsi que l'approximation de $y(t)$ obtenue par RK4. Commenter brièvement les résultats.

3 Question 2

Cette partie est **théorique**. Justifiez en détails vos réponses dans le rapport.

On vous remet la table suivante de vitesses $v(t)$ mesurées à différents temps t

t_i (s)	$v(t_i)$ (m/s)
0	2
1	π
4	e
5	-1

- j) Estimez $v(3.5)$ à l'aide du **meilleur** polynôme d'interpolation de degré 1.
- k) En utilisant le point d'interpolation de $v(t)$ en $t = 1$, déterminer une estimation de l'erreur $E_1(t) = |f(t) - p_1(t)|$, pour tout t où $p_1(t)$ est votre polynôme d'interpolation de la sous-question précédente.
- l) Estimez l'erreur d'interpolation de votre polynôme de degré 1 en $t = 3.5$.
- m) Si on voulait estimer l'accélération en $t = 1$, soit $a(1)$, combien d'approximations de $a(1)$ pouvez-vous faire en appliquant les formules de différences finies vues dans le cours ? Indiquer les formules et valeurs de h possibles.
- n) Estimez $a(1)$. Vous devrez choisir la meilleure formule de différentiation numérique selon l'information donnée. Justifiez votre choix.
- o) Comparez votre dernière réponse avec $p'_1(1)$, où p_1 est le polynôme d'interpolation de degré 1 construit avec $t = 0$ et $t = 1$. Que pouvez-vous en conclure ?
- p) Calculez $p'_2(1)$, où p_2 est le polynôme d'interpolation de degré 2 construit avec $t = 0, 1, 4$. Expliquer pourquoi il diffère des formules d'ordre 2 d'approximation de la dérivée. **Indice** : regarder la distance entre les points d'interpolations par rapport aux distances dans les formules d'approximation de la dérivée.
- q) Si la valeur exacte est $a(1) = 1.1651$, comparez avec $p'_1(1)$ et $p'_2(1)$. Que pouvez-vous en conclure ?
- r) Estimez le déplacement total $D(t) = \int_0^t v(s)ds$ en $t = 5$ (en choisissant la meilleure quadrature de Newton-Cotes selon l'information que vous avez)